

Prompt Iteration Log — tailoring a job-application cover letter

Alex (Xiaoyu) Ji · FlyRank AI-Fluency · FL-05 · 2026-07-13 **Real task (from my FL-01 audit, Target 1):** *draft a tailored cover letter / recruiter outreach for an AI/ML engineer role — send-ready, every claim true, in my voice.*

Six versions. Each iteration adds **one named technique** and I note the **observed change in the output**, not just the prompt.

v0 — Naive (what I'd have typed a month ago)

Prompt: Write a cover letter for this AI/ML engineer job.

Output (excerpt):

Dear Hiring Manager, I am writing to express my strong interest in the AI/ML Engineer position. I am a results-driven engineer with a passion for artificial intelligence and a proven track record of delivering impactful solutions. I am confident my skills make me an excellent fit... Thank you for your consideration.

Could be literally anyone. Buzzwords, zero specifics, no fit to any job.

v1 — technique: Role assignment

Prompt: You are an expert cover-letter writer who specializes in AI/ML engineering roles at startups. Write a cover letter for this AI/ML engineer job.

Note (observed output difference):

- The **tone** got more assured and it self-organized into a clean 3-paragraph shape.
- But the **substance didn't move** — still "proven track record," still no real facts, because a role with nothing to work from just produces *more polished emptiness*.
- *Honest read:* role assignment bought **tone, not truth** — a smaller gain than I expected.
- *Next:* give it real material.

v2 — technique: Context and motivation

Prompt adds: my real facts (*U.S. Navy vet; MS CS 4.0; Orallea — 54★, 1,300+ tests; 11-agent open-source stack; Shadow compliance council*), the **actual JD**, and the motivation (*the reader is a startup hiring manager who skims for ~20 seconds; the goal is to earn a 15-min call, not to list everything*).

Output (excerpt):

...I build production multi-agent systems solo. My trading OS, Orallea, has 54 stars and 1,300+ tests; I cut its per-analysis cost 10x with dual-tier model routing...

Note (observed output difference):

- It **swapped generic adjectives for concrete artifacts** (the 54★ repo, the routing win) and started answering the *actual JD*.

- The letter **could now only be about me** — the biggest single jump.
- *Still failing*: it name-drops projects but doesn't yet map them to what *this* JD asks for; reads like a brag list.
- *Next*: show it what a good, evidence-first claim looks like.

v3 — technique: Few-shot examples ⚠️ partial backfire

Prompt adds: a good-vs-bad example — ❌ "I am passionate about scalable ML." → ✅ "I cut per-analysis LLM cost 10x (0.05→0.005) with no quality drop on a 47/50 blind review."

Note (observed output difference):

- The **shape of the claims improved** — sentences became evidence-first (number + result), matching the ✅ example.
- **But it over-copied my example**: two sentences reused the "0.05→0.005 / 47/50" line almost verbatim as filler even where the JD didn't care, and it echoed my example's cadence instead of the job's needs.
- *Honest read*: few-shot fixed **style** but caused **mimicry** — the model anchored on my sample's content, not just its form.
- *Next*: give it a skeleton so each slot pulls from the JD, not from my example.

v4 — technique: Output structure

Prompt adds: *Structure: (1) one-line hook tied to the company's problem; (2) two proof paragraphs, each mapping ONE of my projects to a JD requirement; (3) one line on veteran/fit; (4) a specific CTA. Hard limit: one page.*

Note (observed output difference):

- Output became **complete and skimmable** — every paragraph now has a job, and the one-page cap killed the padding.
- The **v3 mimicry faded**: because each section had a defined slot tied to a JD requirement, it stopped recycling my example line and started pulling the relevant project per requirement.
- *Still failing*: it picked which project→requirement mappings itself, and once paired a weakly-related project to a requirement to fill the slot.
- *Next*: force it to decide the mappings explicitly before writing.

v5 — technique: Step decomposition

Prompt adds: *First, list the JD's top 5 requirements/keywords. Second, map each to ONE of my real projects (or mark "no strong match"). Third, only then write the letter, using those mappings and nothing invented.*

Output (excerpt):

Mapping: model-routing/cost → Orallexa dual-tier router · multi-agent orchestration → Orallexa 5-voice debate · production/ship → 11-agent stack (PyPI) · regulated/reliability → Shadow audit chain · "CUDA kernels" → *no strong match* (*won't claim it*) Dear [Team], ...

Note (observed output difference):

- Forcing extract → map → write made **every claim traceable to a JD keyword**, and it **honestly flagged the one requirement I don't match** instead of bluffing it.
 - Tailoring became **visible and checkable** — I can see why each sentence is there.
 - *Still failing*: nothing broke; the letter is now send-ready after a light voice pass.
 - *Next*: lock it into a reusable template.
-

Cross-model comparison — same v5 prompt on Claude vs ChatGPT (real run)

I pasted the identical filled prompt (Oracle OVIP AI-Agent Intern JD) into both models. Screenshots: [cross-model-claude.png](#) · [cross-model-chatgpt.png](#). What actually differed:

Dimension	Claude	ChatGPT
The "never bluff a match" rule	Marked "RAG pipelines → no strong match" and refused to claim it: "Shadow's MCP tools are adjacent, but I won't claim RAG — the letter stays silent rather than bluffing."	Bluffed it — folded RAG into "LLMs / RAG / agent workflows → Orallexa," as if Orallexa were a RAG build. Skipped the guardrail entirely.
Mapping-step structure	One clean mapping table, then a note choosing the two strongest proof paragraphs ("cost routing folds into (a)")	Two overlapping tables (keyword→match and requirement→fact) that partly repeat each other
Prioritization	Explicitly picked the 2 strongest angles and merged the rest	Listed every mapping flat, no prioritization
Numbers	Exact, and surfaced test counts (536, 1,300+)	Exact where shown (10x, 0.05→0.005, 47/50)

Verdict (specific, not "both fine"): the decisive difference was **honesty under instruction**. Told to "never bluff a match," **Claude obeyed** and openly flagged RAG as unmatched; **ChatGPT quietly bluffed it** by attaching RAG to a project that isn't RAG. For a cover letter that matters — a false "I do RAG" claim is exactly what collapses in an interview. ChatGPT organized more on the surface (two tables) but skipped the one guardrail that protected me. For this task I'd send Claude's draft after a light pass; I'd trust ChatGPT's only after checking it hadn't invented a match.

Final reusable template (no personal context baked in)

```
You are an expert cover-letter writer for [FIELD] roles at [COMPANY TYPE].

MY FACTS (use ONLY these; never invent):
[paste 4-6 real, verifiable achievements with numbers]

THE JOB:
[paste the JD]

MOTIVATION: the reader is a [hiring manager/recruiter] who skims for ~20 seconds.
The goal is to earn [the one action, e.g. a 15-min call] — not to list everything.

DO THIS IN ORDER (step decomposition):
1. List the JD's top 5 requirements/keywords.
2. Map each to ONE of my facts, or mark "no strong match" (never bluff a match).
3. Only then write the letter, using those mappings and nothing invented.

STRUCTURE (output):
(1) one-line hook tied to the company's problem
(2) two proof paragraphs — each maps ONE fact to ONE JD requirement
(3) one line on personal fit
(4) a specific CTA
HARD LIMIT: one page.

VOICE: direct and plain — evidence over adjectives. Ban "results-driven,"
"passionate," "proven track record." Use my exact numbers, never rounded.

GOOD vs BAD claim (few-shot, for STYLE not content):
❌ "I am passionate about scalable ML."
✅ "[a real metric + the result it produced]"
```